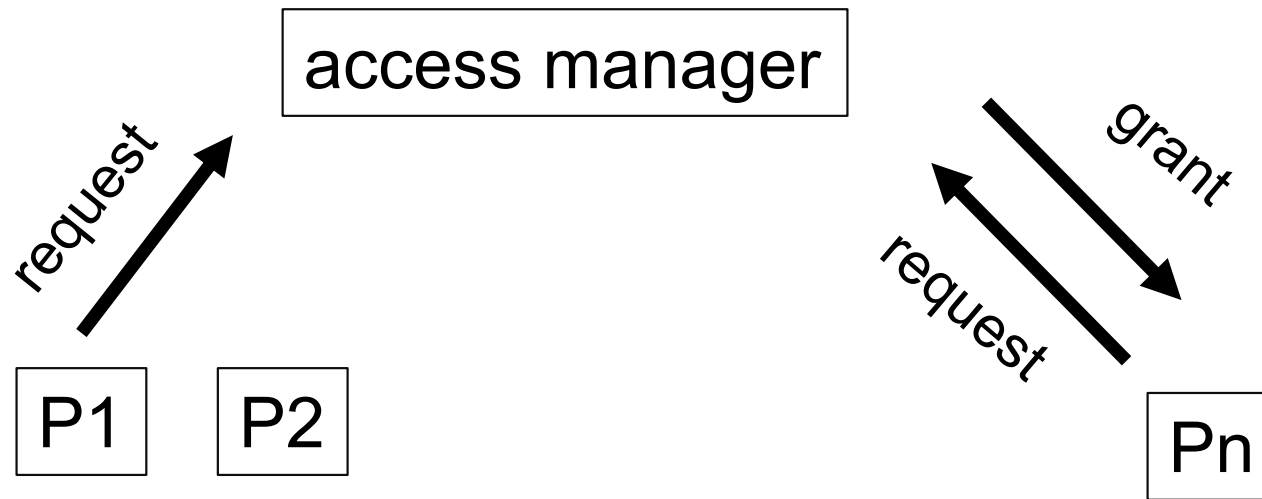


# Mutual Exclusion

- ❑ Mutual exclusion is a fundamental problem in distributed systems
- ❑ Mutual exclusion ensures that concurrent access to the shared resources is **serialised**
  - Only one task is allowed to access the shared resource at a time

# Centralized approach



- ❑ The manager controls the access to the shared resource
- ❑ Problem: single point of failure

- ❑ There are three basic approaches to implement mutual exclusion in distributed systems
  - Timestamp-based
  - Quorum-based
  - Token-based

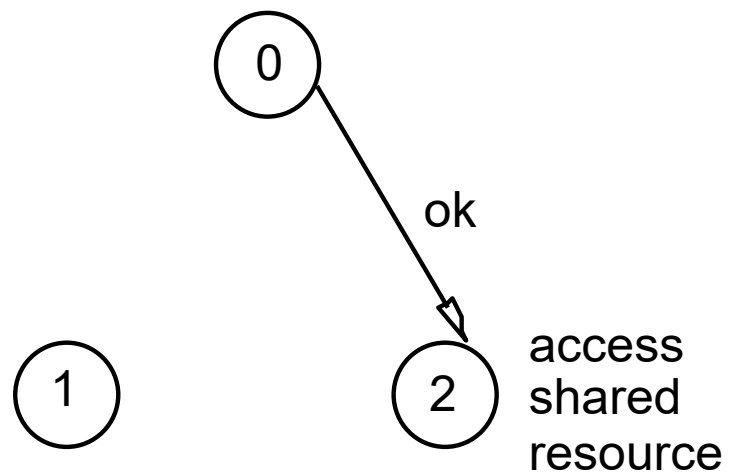
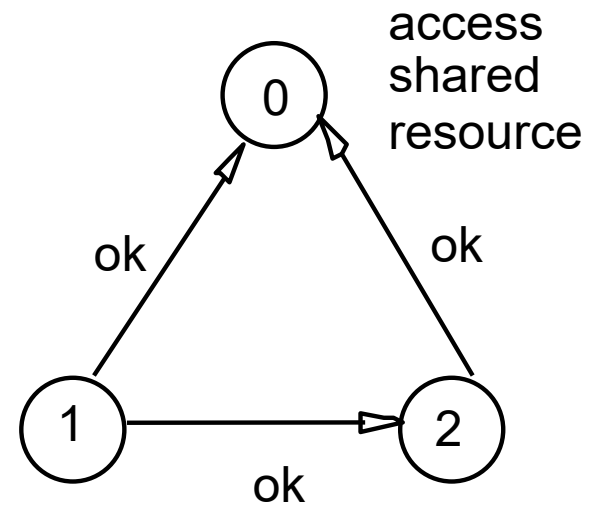
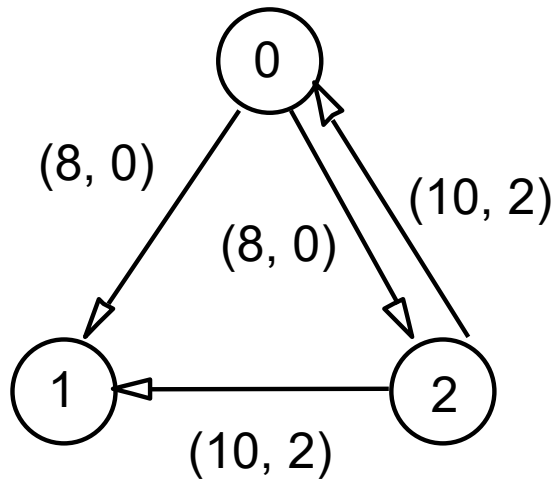
# Requirements for mutual exclusion

- ❑ Safety: only **one** task can access the resource at a time
  - the algorithm does not do anything which is not specified by the original goal
- ❑ Liveness: tasks should not endlessly wait to access the shared resource, i.e., **no deadlock**
- ❑ Fairness: each task gets a **fair chance** to access the shared resource
  - something which need to happen will eventually happen

# Timestamp-based

Ricart et al algorithm

- ❑ The scheme is based on **totally ordered** logical clock
- ❑ **Each request** for accessing the shared resource is given a **timestamp**
- ❑ The requests are **ordered** according to their timestamps
- ❑ The request with the **smallest timestamp** will access the shared resource first



- ❑ The correctness of a distributed algorithm which manages resource in the system normally consists of three aspects.
- ❑ The safeness of Ricart et al algorithm
  - Only one process can enter the critical section at any time.
  - Assume two processes, P1 and P2, want to access the shared resource at the same time. The access requests of the two processes have the timestamp  $(T1, P1)$  and  $(T2, P2)$  respectively, and  $(T1, P1) < (T2, P2)$ .
  - Since  $(T1, P1) < (T2, P2)$ , P1 will **not send OK** message to P2. So, P2 cannot access the shared resource.
  - Thus, P1 and P2 cannot access the shared resource at the same time.

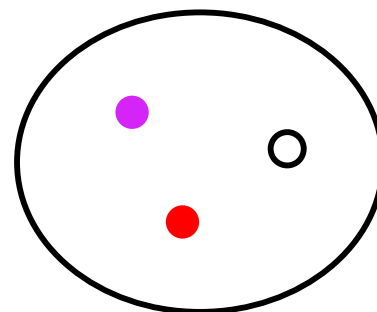
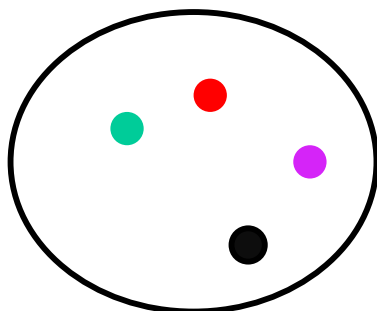
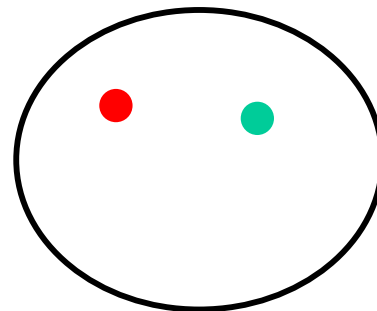
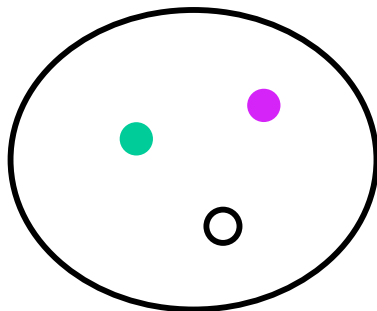
- The liveness of Ricart et al algorithm
  - At least one can access the shared resource
  - A process sends OK to the requester with a smaller timestamp. Since the timestamp is totally ordered timestamp, there is an **order** between any two requests. Thus, the request with the smallest timestamp will be able to access the resource.

## □ The fairness of Ricart et al algorithm

- A process' request will be satisfied eventually.
- According to the algorithm which synchronises the logical clock, the value of the clock **increases** monotonically. Therefore, a process which has waited long enough will have the chance to enter the critical region. Hence, the algorithm is fair.

# Quorum-based

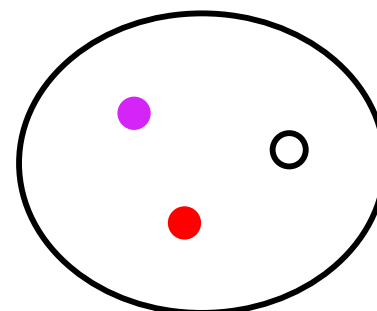
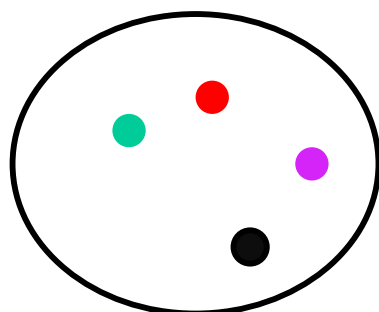
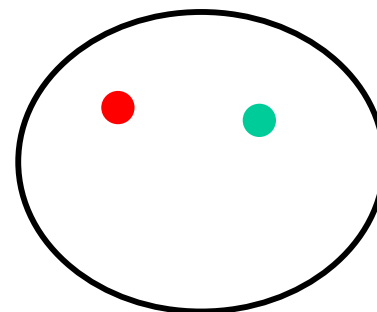
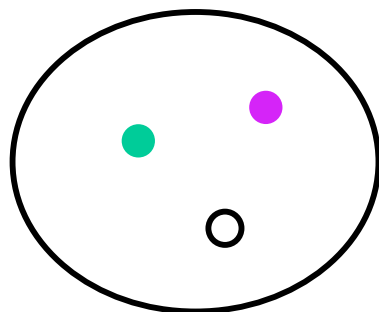
- ❑ In quorum-based algorithms, each task requests permission to access the shared resource from a **subset** of tasks (called a quorum).
- ❑ There are **several quorums** in the system.
- ❑ Each task belong to **one or several** quorums.
- ❑ The quorums must satisfy the following property:
  - The intersection of any two quorums must be **non-empty**.
  - If we have 7 tasks,  $\{1,2,3\}$ ,  $\{2,5,7\}$  and  $\{4,5,6\}$  is **not a valid** assignment of the tasks as  **$\{1,2,3\}$**  and  **$\{4,5,6\}$**  do not intersection with each other.



# Maekawa's algorithm

The main points of the algorithm are as below:

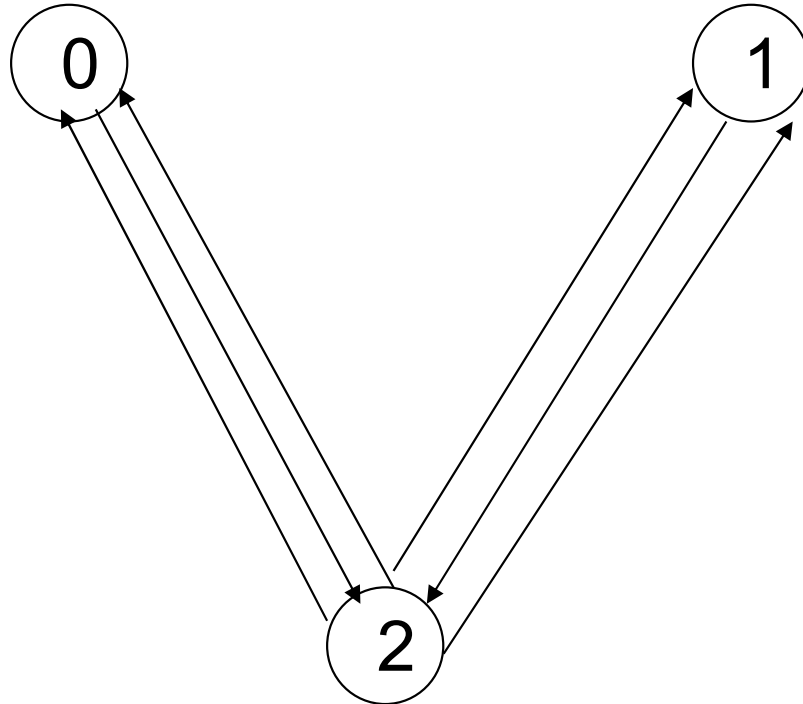
- ❑ There are several **voting sets** in the system
- ❑ Each process has **one vote**.
- ❑ Each process is **assigned** to at least one **voting set**.
- ❑ The voting sets are chosen to **overlap**, and a process must collect **replies from all members** of its voting set.
- ❑ A member can **only reply to one member** of its voting set at a time and will not reply to another request until receiving a **release** or a **relinquish** message.
- ❑ However, it may try to **change its vote** if it receives a request with an earlier timestamp. It does this by sending an **inquire** message to the process it last voted for.



- ❑ When a process receives an **inquire** message
  - if it has already entered the critical section, it discards the message;
  - if it has not already entered the critical section, it decrements its number of votes by one and sends a **relinquish** message to the sender.
- ❑ When a process receives a **relinquish** message it puts the corresponding request on the queue of requests. It then sends a **vote** to the request with the earlier timestamp.

2. vote

2. vote

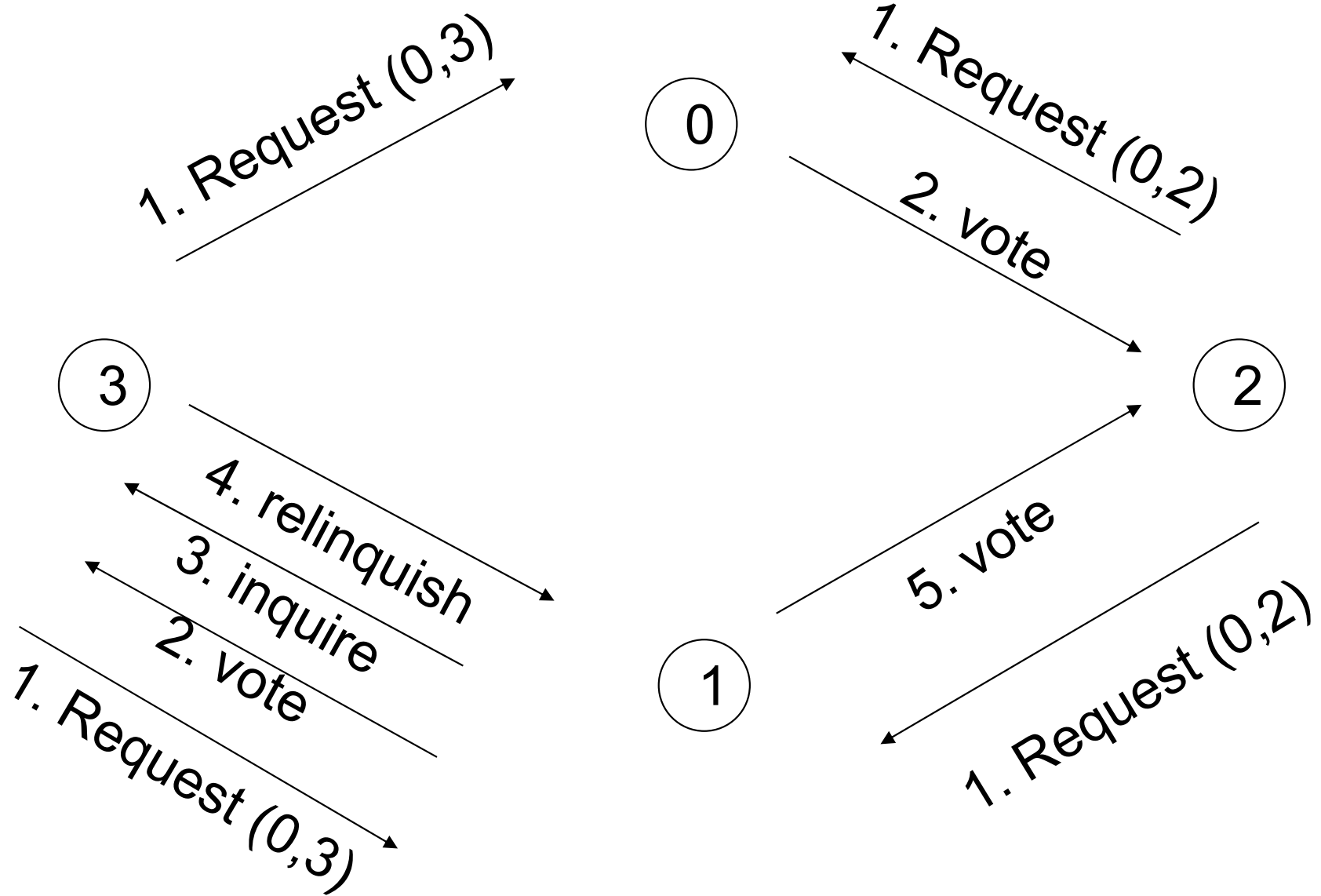


1. request

3. release

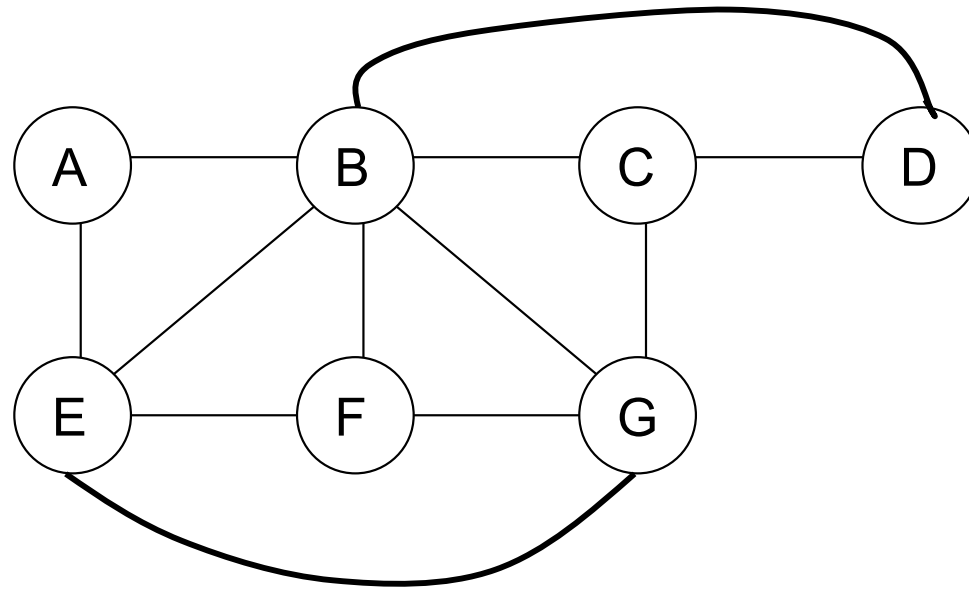
Two voting sets:  $\{0, 1, 3\}$   $\{0, 1, 2\}$

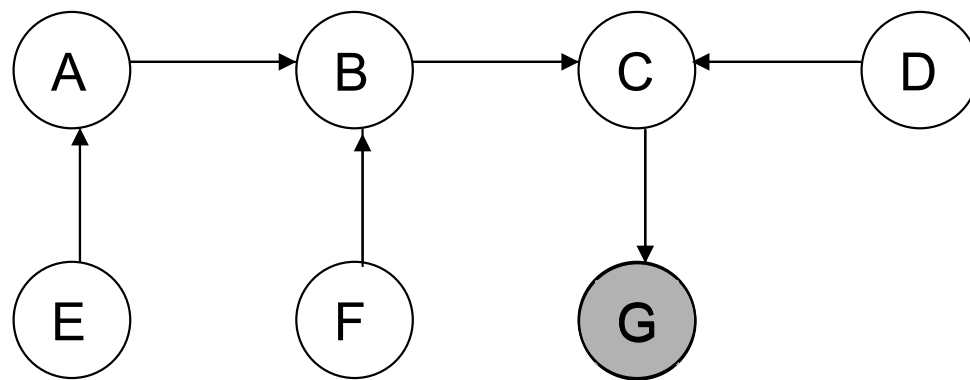
Two voting sets:  $\{0, 1, 3\}$   $\{0, 1, 2\}$



# Token-based

- ❑ In the token-based algorithms, a unique **token** is shared among the tasks.
- ❑ A task is allowed to access the shared resource if it holds the token.
- ❑ In Raymond's algorithm, in order to locate the token, a **spanning tree** is constructed.
  - Each task is a **node** in the tree
  - Each task has a variable, **holder**, that records the identity of one of its neighbours in the spanning tree which might hold the token
  - If a task needs to access the shared resource, the task sends a `token_request` message to the neighbour that might hold the token
  - When receive a `token_request` message, if a task does not have the token, it forwards the request to its neighbour that might hold the token. This process continues until the token is located. The located token is sent to the neighbour that sent the `token_request` message.



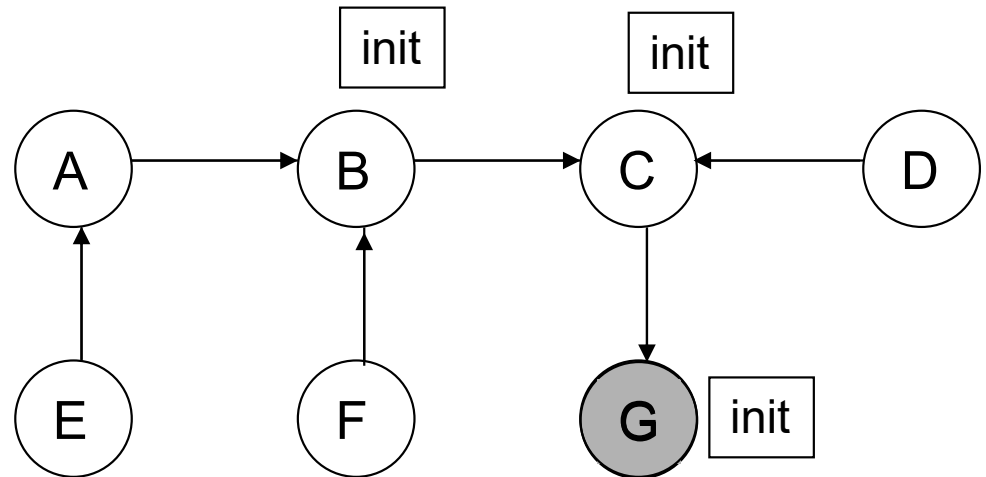
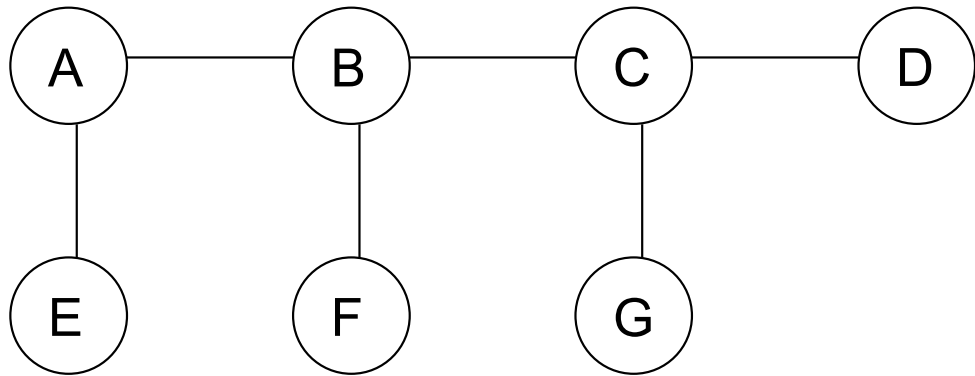


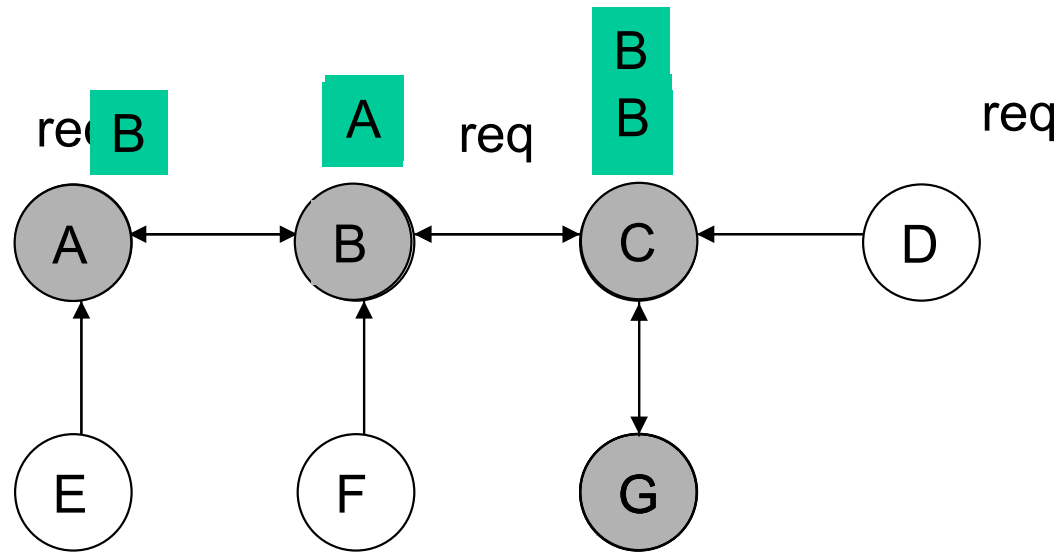
- Each task has a queue, `request_q`, to record the ID of the neighbour from which the `token_request` message is received
- There are two routines for a task to execute
- `Assign_privilege` (execute when a task is holding a token)
  - Check whether the first element in `request_q` is the task itself
  - If it is, remove the first element and access the shared resource. After accessing the resource, run `Assign_privilege` again.
  - If not, sends the token to the neighbour indicated in the first element, removes the first element, and, set holder to the neighbour
- `Make_request` (execute when a task is not holding a token)
  - If the task has not sent any `token_request` message to the neighbour indicated in variable holder, sends `token_request` message to the neighbour indicated in variable holder

- ❑ When a task wants to access the shared resource
  - Add itself to request\_q if it not in the request\_q already
  - If it has the token, run Assign\_privilege
  - Else run Make\_request
- ❑ When a task receives a token\_request message
  - Add the sender to request\_q if the sender is not in request\_q already
  - If the task is holding a token, run Assign\_privilege
  - Else run Make\_request
- ❑ When a task receives a token
  - Run Assign\_privilege
  - If request\_q is not empty, run Make\_request
- ❑ When a task completes using the shared resource
  - Run Assign\_privilege
  - If request\_q is not empty, run Make\_request

## Initialization

- ❑ One task is chosen as the task holding the token
- ❑ The task sends Initialization message to its neighbours
- ❑ The neighbours update variable holder to point to the sender of the Initialization message, and, propagate the Initialization message to their neighbours





# reviews

- ❑ Use pseudo code to describe Ricart and Agrawala's algorithm.
- ❑ Explain why Ricart and Agrawala's algorithm satisfies the safety, liveness and fairness properties.
- ❑ Explain why Maekawa's algorithm satisfies the safety, liveness and fairness properties.
- ❑ Assume that a task might crash in Raymond's algorithm. Augment Raymond's algorithm as described in the lecture to make it tolerate task crash.
  - Hint: after a task recovers from a failure, it can reconstruct its request\_q and holder by asking for the states of its neighbours

- ❑ In Raymond's algorithm, why does the `Make_request` routine only send the `token_request` message to a task's neighbour if the task has not sent a `token_request` message to the neighbour?
- ❑ Assume that you are developing a distributed editing system. The system allows users to edit shared files. Assume that the system does not have a centralised file server. Files are stored on users' local host. Develop a scheme that ensures a file can only be modified by one user at a time.

## reading

- ❑ Ricart, G. and Agrawala, AK(1981). An optimal algorithm for mutual exclusion in computer networks. *Comms. ACM*, vol.24, no.1,pp. 9-17
- ❑ M. Maekawa, A N algorithm for multual exclusion in decentralised systems, *ACM Transaction on Computer Systems*, 3(2), 1995, pp145-159
- ❑ K. Raymond, Tree-based algorithm for distributed mutual exclusion, *ACM Transaction on Computer Systems*, 7, 1989, pp.61-77